SECURE PASSWORD STORAGE SYSTEM

MICROPROJECT REPORT

Submitted by

Anil Kumar — 2411021061563
<Student Name 2> — <Registration Number>
<Student Name 3> — <Registration Number>
<Student Name 4> — <Registration Number>

Submitted in partial fulfilment of the requirements of the course

CRYPTOGRAPHY AND NETWORK SECURITY
Course Code: E1CSA311

Under the Supervision of

Prof. Swetha C B



ALLIANCE SCHOOL OF ADVANCED COMPUTING
ALLIANCE UNIVERSITY, BENGALURU
OCTOBER - 2026

ALLIANCE SCHOOL OF ADVANCED COMPUTING
ALLIANCE UNIVERSITY, BENGALURU
DEPARTMENT OF CSE

CERTIFICATE

This is to certify that the microproject entitled **"SECURE PASSWORD STORAGE SYSTEM"** is a bona fide work carried out by **Anil Kumar – 2411021061563**, **<Student Name 2> – <Registration Number>**, **<Student Name 3> – <Registration Number>**, and **<Student Name 4> – <Registration Number>** as part of the requirements for the course **Cryptography and Network Security (E1CSA311)** during the academic year **2026–2027**, under my guidance.

Faculty in-charge
Prof. Swetha C.B
Assistant Professor
Dept. of CSE

ALLIANCE UNIVERSITY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

We hereby declare that the microproject report entitled **"SECURE PASSWORD STORAGE SYSTEM"** is an original work carried out by us as part of the course **Cryptography and Network Security (E1CSA311)** during the academic year **2026–2027**, under the guidance of **Prof. Swetha C B, Assistant Professor, Alliance School of Advanced Computing, Alliance University**.

We further declare that this work has not been previously submitted, either in whole or in part, for the fulfilment of the requirements of any other course or academic programme. All sources of information used in this work have been appropriately acknowledged and referenced.

Student Name & Registration Number Signature

1. Anil Kumar – 2411021061563 _____
2. <Student Name 2> – <Registration Number> _____
3. <Student Name 3> – <Registration Number> _____
4. <Student Name 4> – <Registration Number> _____

Date: _____

Place: Bengaluru

ABSTRACT

Web applications identify their users with passwords, and those passwords have to be kept in a database so that a login can be checked later. If the database stores the passwords as plain text, or protects them with a weak scheme, a single data breach exposes every account at once. This microproject designs and implements a password storage mechanism in which the original password is never written to disk.

For every account the system generates a random 16-byte salt with `os.urandom()`, appends it to the password and computes the SHA-256 digest of the result with Python's `hashlib` module. Only the username, the salt and the salted hash are stored in an SQLite database. At login, the entered password is hashed again with the stored salt and the two digests are compared in constant time. The system is delivered as a reusable core module with two front ends: a menu-driven command-line program and a Flask web application.

The implementation was verified with 14 automated unit tests and a set of demonstrations. Two users who chose the same password received different salts and therefore different hashes, a precomputed lookup table that recovered passwords from an unsalted database recovered none from the salted one, and changing one character of a password altered 128 of the 256 output bits. A salted hash takes about 0.56 microseconds to compute, so the protection adds no noticeable delay for the user. The report also discusses the main limitation of a single fast hash against offline guessing and proposes key stretching with PBKDF2, bcrypt or Argon2 as future work.

Keywords: *password storage, cryptographic hash function, SHA-256, salt, rainbow table attack, authentication, Python.*

LIST OF SYMBOLS & ABBREVIATIONS

Abbreviation / Symbol	Full Form / Description
CLI	Command-Line Interface
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
CSRF	Cross-Site Request Forgery
DB	Database
FIPS	Federal Information Processing Standards
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
HTTP / HTTPS	Hypertext Transfer Protocol / HTTP Secure
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
PBKDF2	Password-Based Key Derivation Function 2
SHA-256	Secure Hash Algorithm with a 256-bit digest
SQL	Structured Query Language
H(x)	Cryptographic hash of the input x
||	Concatenation of two byte strings
µs, ms	Microsecond, millisecond

CONTENTS

Page No

Title Page.....**i**
Certificate.....**ii**
Declaration.....**iii**
Abstract.....**iv**
List of Symbols & Abbreviations.....**v**

Contents

1. Introduction.....**1**
1.1 Background.....1
1.2 Problem Statement.....1
1.3 Objectives.....1
1.4 Scope of the Microproject.....2
2. System Design and Methodology.....**3**
2.1 Proposed Approach.....3
2.2 System Architecture.....3
2.3 Modules/Components.....5
2.4 Algorithm or Working Procedure.....5
3. Implementation.....**7**
3.1 Software and Hardware Requirements.....7
3.2 Tools and Technologies Used.....7
3.3 Dataset/Input Data.....7
3.4 Implementation Details.....8
4. Results and Discussion.....**11**
4.1 Results/Observations.....11
4.2 Performance Analysis.....12
4.3 Security Analysis.....13
4.4 Screenshots/Sample Outputs.....15
5. Conclusion and Future Scope.....**21**
References.....**22**
Appendix.....**23**

1. INTRODUCTION

1.1 Background

Passwords remain the most common way of proving identity on the web. When a user creates an account, the application has to remember something about the chosen password so that it can recognise the same user at the next login. The simplest design is to save the password itself in a database table. This design is also the most dangerous one: databases are copied in backups, read by administrators and, from time to time, stolen through attacks such as SQL injection or misconfigured servers. If the table contains plain-text passwords, anyone who obtains a copy can log in as any user, and because people reuse passwords, the damage spreads to their accounts on other services as well.

Cryptography offers a better option. A cryptographic hash function such as SHA-256 maps an input of any length to a fixed 256-bit output. It is a one-way function: computing the hash from the password is fast, but there is no practical method of computing the password from the hash. An application can therefore store only the hash and still check a login by hashing the entered password and comparing the result.

Hashing alone is not sufficient. Because the same password always produces the same hash, an attacker can prepare a table of the hashes of millions of common passwords once and then look up every stolen hash in that table. Rainbow tables are a compact form of such precomputed tables. Identical hashes also reveal which users share a password. Both weaknesses are removed by a salt: a random value, unique to each account, that is combined with the password before hashing. The salt does not need to be secret. Its job is to make every stored hash the output of a different computation, so that precomputed tables become useless and equal passwords no longer look equal.

1.2 Problem Statement

In a real-world web application, user accounts are created and maintained by storing credentials in a database. If passwords are stored in plain text or with weak protection, a data breach can expose sensitive user information. The problem addressed in this microproject is to design and build a password storage and verification system that continues to protect user passwords even when the database itself has been compromised.

Specifically, the system must (a) never store or display the original password, (b) resist rainbow-table attacks, (c) produce different stored values for users who happen to choose the same password, and (d) still verify a login correctly and quickly.

1.3 Objectives

- To implement SHA-256 hashing of passwords using the `hashlib` module of Python.

- To generate a unique random salt for every password using `os.urandom()` and append it to the password before hashing.
- To store only the username, the salt and the salted hash in a database, with no plain-text password anywhere in the system.
- To verify a login by re-hashing the entered password with the stored salt and comparing it with the saved hash.
- To demonstrate that two users with the same password receive different hashes, and that a precomputed (rainbow-table) attack fails against the salted database.
- To provide both a command-line interface and a web interface on top of one shared core module.
- To measure the performance of the scheme and analyse its security, including its limitations.

1.4 Scope of the Microproject

The microproject covers user registration, secure storage of credentials, login verification and a set of demonstrations and measurements. It maps to Module III of the syllabus (public-key cryptography and authentication requirements), where hash functions are studied as a building block for authentication, and to course outcome CO3.

The following items are within scope: salt generation, salted SHA-256 hashing, storage in an SQLite database, constant-time verification, input validation, a command-line program, a Flask web application, unit tests, a performance benchmark and a security analysis.

The following items are outside the scope and are discussed only as future work: deliberately slow password-hashing algorithms (PBKDF2, bcrypt, scrypt, Argon2), transport security (HTTPS/TLS), account lockout and rate limiting, multi-factor authentication, password reset by e-mail, and deployment on a production web server.

2. SYSTEM DESIGN AND METHODOLOGY

2.1 Proposed Approach

The system stores, for each user, a record of the form (username, salt, hash), where

```
hash = SHA-256( password || salt )
```

and the salt is a fresh 128-bit random value drawn from the operating system's cryptographically secure random number generator. The password is used only in memory, for the duration of one function call, and is never written to the database, to a file or to the screen. Table 2.1 compares this approach with the two weaker alternatives.

Table 2.1: Comparison of password storage approaches

Property	Plain text	Unsalted SHA-256	Salted SHA-256 (proposed)
What a database thief obtains	Every password, immediately	Hashes that can be looked up in precomputed tables	Salts and hashes; no table applies
Rainbow-table attack	Not needed	Succeeds for common passwords	Defeated
Same password, two users	Visible	Visible (identical hashes)	Hidden (different hashes)
Cost of cracking n users	None	One dictionary pass for all users	One dictionary pass per user
Login verification	String comparison	Hash and compare	Hash with stored salt and compare

Three design decisions follow from this approach. First, the salt is generated with `os.urandom()` and not with the `random` module, because the latter is predictable and not intended for security. Second, the salt is stored next to the hash in the same row; it must be available at login and does not need to be secret. Third, the comparison of the two digests uses `hmac.compare_digest()`, which takes the same time whether the digests differ in the first character or the last, so that response time leaks nothing about the stored hash.

2.2 System Architecture

The system is organised in three layers, shown in Figure 2.1. The interface layer contains two independent front ends. Both call the same core module, which holds all of the cryptographic logic, and the core module is the only component that reads from or writes to the database. Keeping the security logic in one place means that it is written once, tested once and behaves identically in the terminal and in the browser.

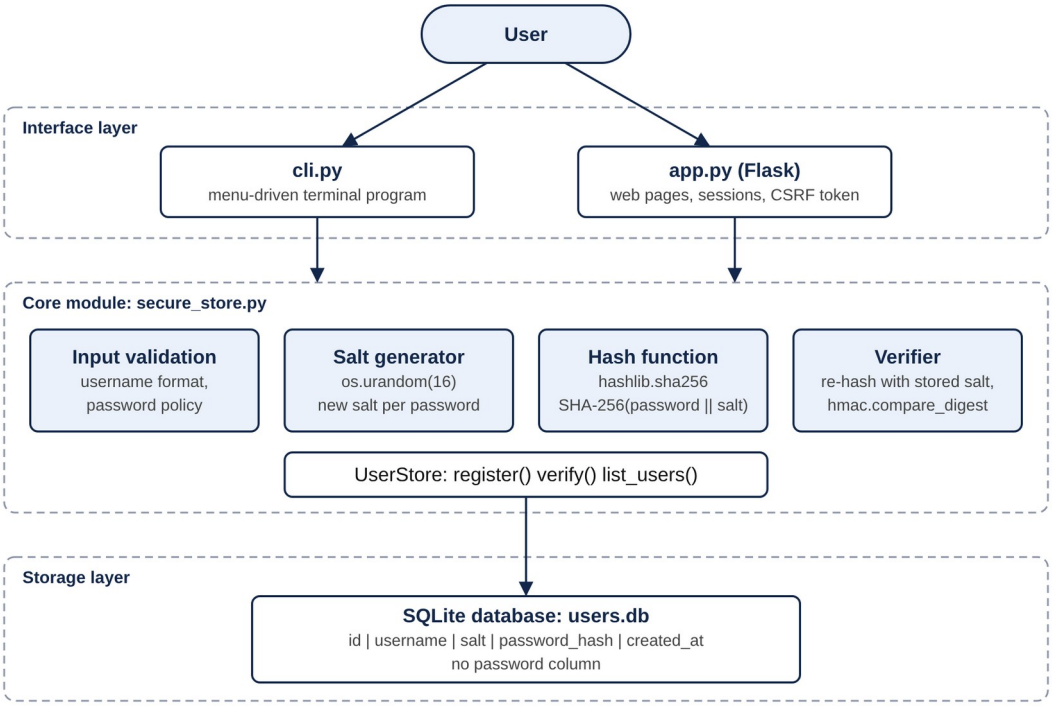


Figure 2.1: Three-layer architecture of the Secure Password Storage System

2.3 Modules/Components

Table 2.2: Modules of the system

Module	File	Responsibility
Salt generator	`secure_store.py`	`generate_salt()` returns 16 random bytes from `os.urandom()`.
Hash function	`secure_store.py`	`hash_password()` returns the hexadecimal SHA-256 digest of password || salt.
Input validation	`secure_store.py`	Checks the username format and the password policy (at least 8 characters, one letter, one digit).
User store	`secure_store.py`	`UserStore` class: creates the table, registers users, verifies logins and lists records through parameterised SQL.
Command-line interface	`cli.py`	Menu for register, login and viewing stored records; passwords are read with `getpass` so they are not echoed.
Web interface	`app.py`, `templates/`	Flask routes for register, login, dashboard, stored records and logout, with a signed session cookie and CSRF token.
Demonstration	`demo.py`	Shows the same-password case, login checks, the rainbow-table attack and the avalanche effect.
Benchmark	`benchmark.py`	Measures salt generation, hashing, registration and login times.
Unit tests	`tests/`	14 automated tests of hashing, storage and verification.

2.4 Algorithm or Working Procedure

Registration

1. Read the username and the password from the user.
2. Validate the username format and the password policy. If either check fails, report the error and stop.
3. Generate a salt: `salt = os.urandom(16)`.
4. Compute `hash = SHA-256(password || salt)` and convert the digest to hexadecimal.
5. Insert (username, salt, hash, creation time) into the `users` table. If the username already exists, the database rejects the row and an error is reported.
6. Return success. The password variable goes out of scope and is not saved anywhere.

Login verification

1. Read the username and the password from the user.

2. Fetch the salt and the stored hash for that username from the `users` table.
3. If no such user exists, compute one dummy hash (so that the response time is similar) and deny access.
4. Compute `candidate = SHA-256(entered password || stored salt)`.
5. Compare `candidate` with the stored hash using `hmac.compare_digest()`.
6. Grant access if they are equal; otherwise deny access with the same message used for an unknown user.

Figure 2.2 shows the two procedures as flowcharts.

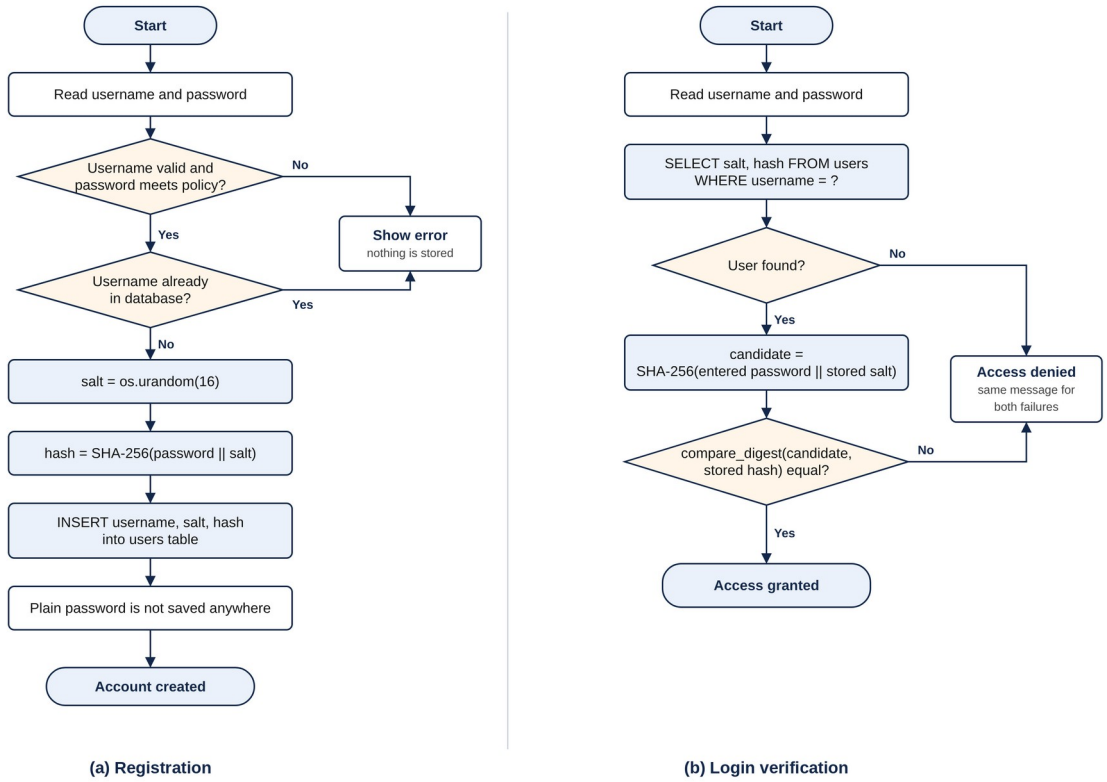


Figure 2.2: Flowcharts of (a) registration and (b) login verification

3. IMPLEMENTATION

3.1 Software and Hardware Requirements

Table 3.1: Software and hardware requirements

Category	Requirement
Operating system	Windows, macOS or Linux
Language	Python 3.8 or later (developed and tested with Python 3.13)
Standard-library modules	`hashlib`, `os`, `hmac`, `sqlite3`, `re`, `getpass`, `unittest`
External package	Flask 3.x (needed only for the web interface)
Editor	Visual Studio Code with the Python extension
Browser	Any modern browser, for the web interface
Processor and memory	Any dual-core processor with 2 GB RAM or more
Disk space	Less than 5 MB for the project and its database

3.2 Tools and Technologies Used

Table 3.2: Tools and technologies

Tool / library	Purpose in this project
`hashlib.sha256`	Computes the SHA-256 digest defined in FIPS 180-4.
`os.urandom`	Provides random bytes from the operating system's CSPRNG for the salt, the session key and the CSRF token.
`hmac.compare_digest`	Compares two digests in constant time.
`sqlite3`	Serverless relational database in a single file (`users.db`).
`getpass`	Reads a password in the terminal without displaying it.
Flask	Lightweight web framework used for the browser interface.
`unittest`	Automated testing of the core module.
Visual Studio Code	Code editing, integrated terminal and debugging.

3.3 Dataset/Input Data

The project does not use an external dataset. Its input is the username and password typed by the user. The database consists of one table whose structure is given in Table 3.3; note that it has no column for the password.

Table 3.3: Structure of the users table

Column	Type	Description
`id`	INTEGER, primary key	Automatically assigned record number
`username`	TEXT, unique	Login name, 3 to 30 characters
`salt`	TEXT	16 random bytes, stored as 32 hexadecimal characters
`password_hash`	TEXT	SHA-256(password || salt), 64 hexadecimal characters
`created_at`	TEXT	Date and time of registration

For the demonstrations and tests, the sample accounts in Table 3.4 were used. Two of them deliberately share a password. The rainbow-table demonstration uses a small list of ten common passwords (for example `123456`, `password`, `qwerty123` and `Summer2026`) to stand in for the very large tables used by real attackers.

Table 3.4: Sample accounts used for testing

Username	Password	Purpose
alice	Summer2026	Normal registration and login
bob	Summer2026	Same password as alice, to show that the hashes differ
charlie	Tr1cky#Pass	A password that is not in the attacker's list
mallory	(not registered)	Login attempt for a user that does not exist

3.4 Implementation Details

Salt generation and hashing. These two functions are the heart of the system. The password is encoded as UTF-8 bytes, the salt is appended and the SHA-256 digest of the combined bytes is returned in hexadecimal form.

```
def generate_salt(length=SALT_BYTES):
    """Return `length` cryptographically secure random bytes."""
    return os.urandom(length)


def hash_password(password, salt):
    """Return the hex SHA-256 digest of the password with the salt appended."""
    return hashlib.sha256(password.encode("utf-8") + salt).hexdigest()
```

Registration. The method validates the input, generates the salt, computes the hash and inserts the record. The SQL statement uses `?` placeholders, so user input is always passed as data and can never change the meaning of the query. The `UNIQUE` constraint on the username column makes the database itself reject a duplicate.

```
    def register(self, username, password):
        """Create an account. Returns (ok, message)."""
        username = (username or "").strip()
        error = check_username(username) or check_password(password)
        if error:
            return False, error

        salt = generate_salt()
        digest = hash_password(password, salt)
        try:
            with self._connect() as con:
                # parameterised query -> no SQL injection
                con.execute(
                    "INSERT INTO users (username, salt, password_hash, created_at) "
                    "VALUES (?, ?, ?, ?)",
                    (username, salt.hex(), digest,
                     datetime.now().strftime("%Y-%m-%d %H:%M:%S")),
                )
        except sqlite3.IntegrityError:
            return False, "That username is already taken."
        return True, "Account created."
```

Login verification. The stored salt is read, the entered password is hashed with it and the result is compared with the stored hash.

```
    def verify(self, username, password):
        """Return True only if the username exists and the password matches."""
        with self._connect() as con:
            row = con.execute(
                "SELECT salt, password_hash FROM users WHERE username = ?",
                ((username or "").strip(),),
            ).fetchone()

        if row is None:
            # Do the same amount of work for an unknown user, so response
            # time does not reveal which usernames exist.
            hash_password(password or "", generate_salt())
            return False

        salt = bytes.fromhex(row["salt"])
        candidate = hash_password(password or "", salt)
        # constant-time comparison prevents timing attacks on the digest
        return hmac.compare_digest(candidate, row["password_hash"])
```

Command-line interface. `cli.py` presents a four-option menu (register, login, view stored records, exit). Passwords are read with `getpass()`, so they do not appear on the screen, and registration asks for the password twice to catch typing mistakes. The option to view stored

records prints the salt and hash of every user, which makes it easy to confirm that no password is kept.

Web interface. `app.py` is a Flask application that calls the same `UserStore` class. It offers pages for registration, login, a dashboard that is shown only after a successful login, and a demonstration page that lists the stored records. After a successful login the session is cleared and re-created, the session cookie is signed with a random key and marked HttpOnly, and every form carries a random CSRF token that is checked on submission. A failed login always shows the same message, *Invalid username or password*, whether the username or the password was wrong.

Testing. `tests/test_store.py` contains 14 unit tests. They check the hash against the published SHA-256 test vector for the string “abc”, the length and uniqueness of salts, successful and failed logins, rejection of duplicate usernames and weak passwords, the same-password case, an SQL-injection attempt, and that the plain-text password does not occur anywhere in the bytes of the database file.

4. RESULTS AND DISCUSSION

4.1 Results/Observations

The system was exercised through both interfaces and through the automated tests. Table 4.1 summarises the functional test cases. All of them produced the expected result, and all 14 unit tests passed (Figure 4.8).

Table 4.1: Functional test cases and results

No.	Test case	Expected result	Observed
1	Register a new user with a valid password	Account created; salt and hash stored	As expected
2	Register with a username that already exists	Rejected: username already taken	As expected
3	Register with a password shorter than 8 characters or without a digit	Rejected with a policy message	As expected
4	Login with the correct password	Access granted	As expected
5	Login with a wrong password (including wrong letter case)	Access denied	As expected
6	Login with a username that is not registered	Access denied, same message as case 5	As expected
7	Two users register with the same password	Different salts and different hashes	As expected
8	Search the database file for the plain-text password	Not found	As expected
9	SQL-injection string entered as username or password	Access denied; table unchanged	As expected
10	Precomputed table of common-password hashes applied to the salted database	No password recovered	As expected

Same password, different hashes. In the demonstration run, alice and bob both registered with the password `Summer2026`. The values actually stored were:

```
alice  salt = f9abf820752b3fb63a540dcee89d59ca
       hash = 4342cbc34610ec18f77247b19c2aafe6eda7f20c1cf5faf13ee78fb932ae466a
bob    salt = cbfa2e1d7004efe3ab797f2930f90e24
       hash = 816a1b5e187951ed11cc5001276482bcc1a1c84c8930286189264d6fb415c740
```

The two hashes have nothing in common, so someone reading the database cannot tell that the passwords are equal. Without a salt, both rows would have contained the same value, `ed3c9886c697c0ac…07043d1a`.

Rainbow-table attack. A lookup table of the unsalted SHA-256 hashes of ten common passwords was built once. Applied to an unsalted copy of the database, it recovered the passwords of alice and bob immediately; charlie's password was not in the list. Applied to the salted database produced by this system, it recovered none of the three (Figure 4.7).

Avalanche effect. Hashing `Summer2026` and `Summer2027` with the same salt gave digests that differ in 128 out of 256 bits (50.0%). A small change in the password gives a completely different hash, so a stored hash provides no hint about how close a guess is.

4.2 Performance Analysis

Timings were measured with `benchmark.py` using `time.perf_counter()` on Python 3.13 (Linux, x86-64, 2.1 GHz Intel Xeon). Each primitive operation was repeated 20,000 times per round and the best of 15 rounds was taken, which filters out interference from other processes. Absolute values will differ from one machine to another; the script can be re-run on any computer to obtain its own figures.

Table 4.2: Measured execution times

Operation	Average time
Salt generation, `os.urandom(16)`	0.439 µs
SHA-256 without salt	0.531 µs
SHA-256 with 16-byte salt	0.556 µs
Registration (salt + hash + SQLite INSERT)	1.128 ms
Login (SQLite SELECT + hash + comparison)	0.082 ms
PBKDF2-HMAC-SHA256, 600,000 iterations (for comparison)	176.9 ms

Salting adds about 0.02 µs to a hash and 0.44 µs to generate, which is negligible. The time of a complete registration or login is dominated by the database, not by the cryptography: registration takes about 1.1 ms because the row has to be committed to disk, while login needs only a read and takes about 0.08 ms.

Figure 4.1 shows the hashing time for passwords of 8 to 256 characters. The time is almost constant up to 32 characters, because the password and salt together still fit in a single 64-byte SHA-256 block, and rises only slightly for longer inputs (from 0.57 µs to 0.74 µs). Long passwords therefore carry no performance penalty.

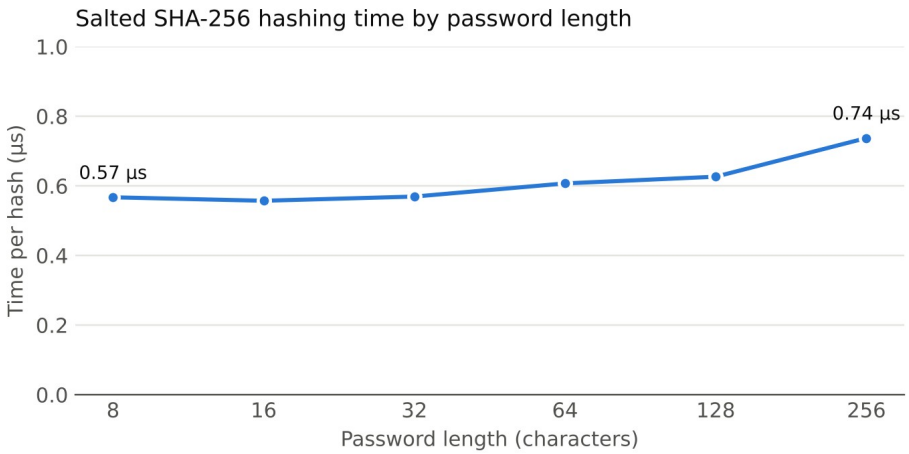


Figure 4.1: Salted SHA-256 hashing time for different password lengths

The benefit of the salt appears on the attacker's side. Table 4.3 and Figure 4.2 show the number of hash computations needed to test a dictionary of one million candidate passwords against a stolen database. Without salts, one pass over the dictionary (or one precomputed table) serves every account. With per-user salts, the whole dictionary has to be hashed again for each account, so the work grows in proportion to the number of users.

Table 4.3: Hash computations needed to test a 1,000,000-word dictionary

Accounts in database	Unsalted SHA-256	Salted SHA-256	Increase
1	1,000,000	1,000,000	1×
100	1,000,000	100,000,000	100×
10,000	1,000,000	10,000,000,000	10,000×

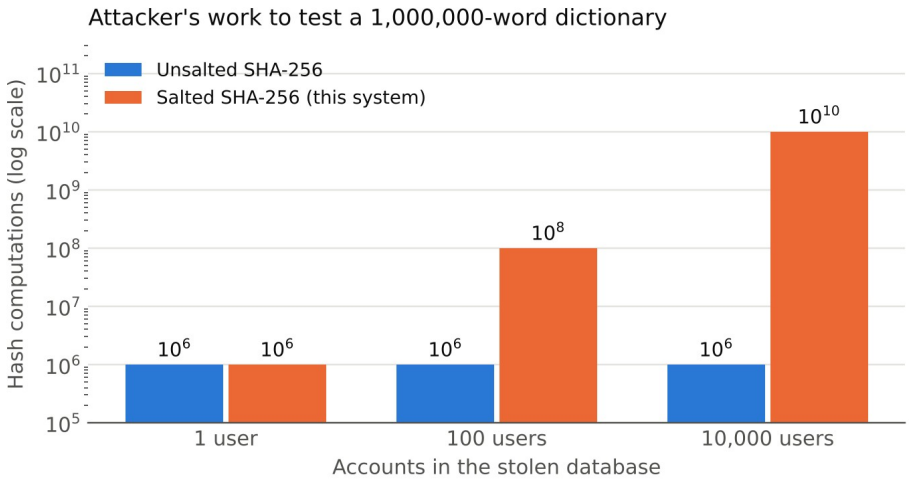


Figure 4.2: Attacker's work with and without salts (logarithmic scale)

4.3 Security Analysis

Table 4.4 lists the threats considered and how the system responds to each.

Table 4.4: Threats and countermeasures

Threat	Countermeasure in this system
Database theft exposes passwords	Only salts and SHA-256 hashes are stored. SHA-256 is preimage resistant, so the password cannot be computed from the hash.
Rainbow-table or other precomputed attack	A 128-bit random salt per password. A table would have to be built separately for each of 2^{128} possible salts, which is infeasible.
Identifying users who share a password	Different salts give different hashes for equal passwords.
Predictable salts	Salts come from `os.urandom()`, the operating system's CSPRNG, not from the `random` module.
Timing attack on the comparison	`hmac.compare_digest()` compares in constant time.
Username enumeration	The same error message, and a dummy hash computation, for unknown users and wrong passwords.
SQL injection	All queries are parameterised; verified by a unit test.
Shoulder surfing and password display	`getpass` in the terminal and password fields in the browser; passwords are never printed or logged.
Weak passwords	Minimum length of 8 characters with at least one letter and one digit.
Cross-site request forgery and session fixation (web)	Random CSRF token in every form; session re-created at login; signed, HttpOnly cookie.

Limitations. The analysis also has to state what salted SHA-256 does not protect against. SHA-256 is designed to be fast, and a salt does not slow down an attacker who targets one account: knowing the salt, the attacker can simply hash guesses for that user.

The benchmark measured about 1,797,562 salted hashes per second on a single processor core in Python. At that rate, trying every 8-character password made of lowercase letters and digits ($36^8 \approx 2.82 \times 10^{12}$ candidates) would take about 18 days, and password-cracking tools running on graphics processors compute SHA-256 several orders of magnitude faster than that. A weak or short password is therefore still at risk after a breach, even though it is salted.

The accepted remedy is key stretching: replacing the single hash with a deliberately slow function. As a comparison, PBKDF2-HMAC-SHA256 with 600,000 iterations took 176.9 ms per password on the same machine, roughly 318,000 times slower than one SHA-256. That delay is unnoticeable to a user logging in once, but it multiplies the attacker's cost by the same factor. This is listed as the first item of future work.

Two further limitations are that the web interface runs over plain HTTP on the local machine, so a deployed version would need HTTPS to protect the password in transit, and that there is no limit on the number of login attempts, so online guessing is not slowed down.

4.4 Screenshots/Sample Outputs

The figures in this section are the outputs produced by the programs. Figures 4.3 to 4.5 show the command-line interface, Figures 4.6 to 4.9 the demonstration, test and benchmark scripts, and Figures 4.10 to 4.14 the web interface.

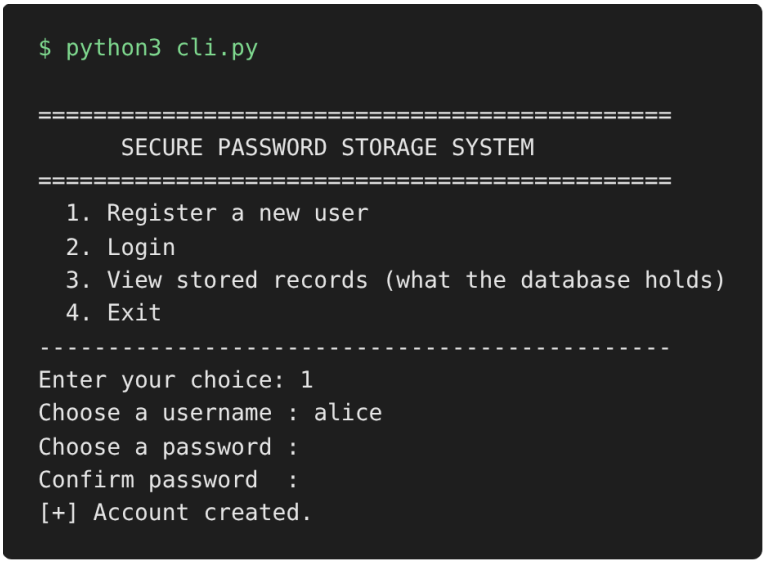


Figure 4.3: Command-line interface: main menu and registration (the password is not echoed)

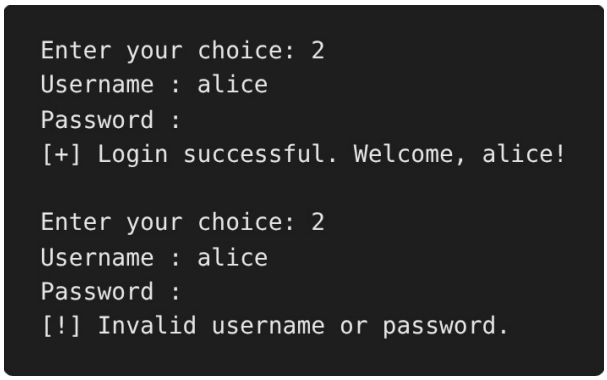


Figure 4.4: Command-line interface: successful and failed login

```
Enter your choice: 3

User #1: alice   (created 2026-10-03 21:38:57)
  salt : bf4a45a31e2d6283af7f913f77cbb2c8
  hash : 5a948c200b4d94ae1d074b250785ed6e16c7efa4c51ae7f9a8dc56dce30462f3

User #2: bob   (created 2026-10-03 21:38:58)
  salt : 6eea882c8ab7e81194e45010d8f0632c
  hash : 82dd78fd63b7c196ce3c128228b3f84bf8919e555f4c90058e4c17955961b750

Note: the original passwords are not stored anywhere.
```

Figure 4.5: Command-line interface: stored records for two users with the same password

```
$ python3 demo.py
--------------------------------------------------------------------------------
DEMO 1: Registration stores a salt and a hash, never the password
--------------------------------------------------------------------------------
register('alice', 'Summer2026') -> Account created.
register('bob', 'Summer2026') -> Account created.
register('charlie', 'Tr1cky#Pass') -> Account created.

alice    salt=f9abf820752b3fb63a540dcee89d59ca
         hash=4342cbc34610ec18f77247b19c2aafe6eda7f20c1cf5faf13ee78fb932ae466a
bob      salt=cbfa2e1d7004efe3ab797f2930f90e24
         hash=816a1b5e187951ed11cc5001276482bcc1a1c84c8930286189264d6fb415c740
charlie  salt=615bcb900741bbd491af919740388dd1
         hash=4fa3960ff365f0bfd1675e812f2dc2ae1b13cda70d5fe66c48e8dc2a8bbf1a6f

--------------------------------------------------------------------------------
DEMO 2: Same password, different salts -> different hashes
--------------------------------------------------------------------------------
alice and bob both chose the password 'Summer2026'
alice hash : 4342cbc34610ec18f77247b19c2aafe6eda7f20c1cf5faf13ee78fb932ae466a
bob   hash : 816a1b5e187951ed11cc5001276482bcc1a1c84c8930286189264d6fb415c740
hashes identical? -> False

Without a salt both would be stored as:
             ed3c9886c697c0ac147b04cf350c8a8c00a71ab10e1439904b80e2c307043d1a
```

Figure 4.6: demo.py: stored salts and hashes, and the same-password case

```
------------------------------------------------------------------------------
DEMO 3: Login verification by re-hashing with the stored salt
------------------------------------------------------------------------------
login('alice', 'Summer2026') -> ACCESS GRANTED
login('alice', 'summer2026') -> ACCESS DENIED
login('bob', 'WrongPass1') -> ACCESS DENIED
login('mallory', 'Summer2026') -> ACCESS DENIED

------------------------------------------------------------------------------
DEMO 4: Rainbow-table (precomputed hash) attack
------------------------------------------------------------------------------
Attacker precomputes SHA-256 of 10 common passwords.

(a) Against a database of UNSALTED hashes:
    alice    -> CRACKED: 'Summer2026'
    bob      -> CRACKED: 'Summer2026'
    charlie  -> not in table

(b) Against this system's SALTED hashes (same lookup table):
    alice    -> not in table
    bob      -> not in table
    charlie  -> not in table

    Passwords recovered from the salted database: 0

------------------------------------------------------------------------------
DEMO 5: Avalanche effect of SHA-256
------------------------------------------------------------------------------
SHA-256('Summer2026' + salt) = 4342cbc34610ec18f77247b19c2aafe6eda7f20c1cf5faf13ee78fb932ae466a
SHA-256('Summer2027' + salt) = 9add4499c05af8c3df59ae538fbc263528d0db803f3bcbce4bd45b19b9a7fca1
One character changed -> 128 of 256 output bits differ (50.0%)
```

Figure 4.7: demo.py: login verification, rainbow-table attack and avalanche effect

```
$ python3 -m unittest discover -s tests -v
test_known_sha256_vector (test_store.HashTests.test_known_sha256_vector) ... ok
test_new_salt_new_hash (test_store.HashTests.test_new_salt_new_hash) ... ok
test_salt_length (test_store.HashTests.test_salt_length) ... ok
test_salts_unique (test_store.HashTests.test_salts_unique) ... ok
test_same_salt_same_hash (test_store.HashTests.test_same_salt_same_hash) ... ok
test_bad_username (test_store.StoreTests.test_bad_username) ... ok
test_duplicate_username (test_store.StoreTests.test_duplicate_username) ... ok
test_no_plaintext_in_db_file (test_store.StoreTests.test_no_plaintext_in_db_file) ... ok
test_register_and_login (test_store.StoreTests.test_register_and_login) ... ok
test_same_password_two_users (test_store.StoreTests.test_same_password_two_users) ... ok
test_sql_injection (test_store.StoreTests.test_sql_injection) ... ok
test_unknown_user (test_store.StoreTests.test_unknown_user) ... ok
test_weak_password (test_store.StoreTests.test_weak_password) ... ok
test_wrong_password (test_store.StoreTests.test_wrong_password) ... ok

----------------------------------------------------------------------
Ran 14 tests in 0.027s

OK
```

Figure 4.8: All 14 unit tests passing

```
$ python3 benchmark.py
PERFORMANCE ANALYSIS
============================================================

1. Primitive operations (average of 20,000 runs, best of 15 rounds)
   os.urandom(16) salt generation :    0.439 us
   SHA-256 without salt           :    0.531 us
   SHA-256 with 16-byte salt      :    0.556 us

2. Salted SHA-256 time against password length
      8 characters :  0.567 us
     16 characters :  0.557 us
     32 characters :  0.569 us
     64 characters :  0.607 us
    128 characters :  0.626 us
    256 characters :  0.737 us

3. End-to-end operations with SQLite (average of 200)
   register (salt + hash + INSERT) :  1.128 ms
   login    (SELECT + hash + compare):  0.082 ms

4. Attacker's work to test a dictionary against a stolen database
   measured single-core rate: 1,797,562 hashes per second
        1 user(s), 1M-word dictionary: unsalted      1,000,000 hashes | salted         1,000,000 hashes
      100 user(s), 1M-word dictionary: unsalted      1,000,000 hashes | salted       100,000,000 hashes
    10000 user(s), 1M-word dictionary: unsalted      1,000,000 hashes | salted    10,000,000,000 hashes

5. For comparison: PBKDF2-HMAC-SHA256, 600,000 iterations
   one hash : 176.9 ms (about 317,917x slower than a single SHA-256)
```

Figure 4.9: Output of benchmark.py

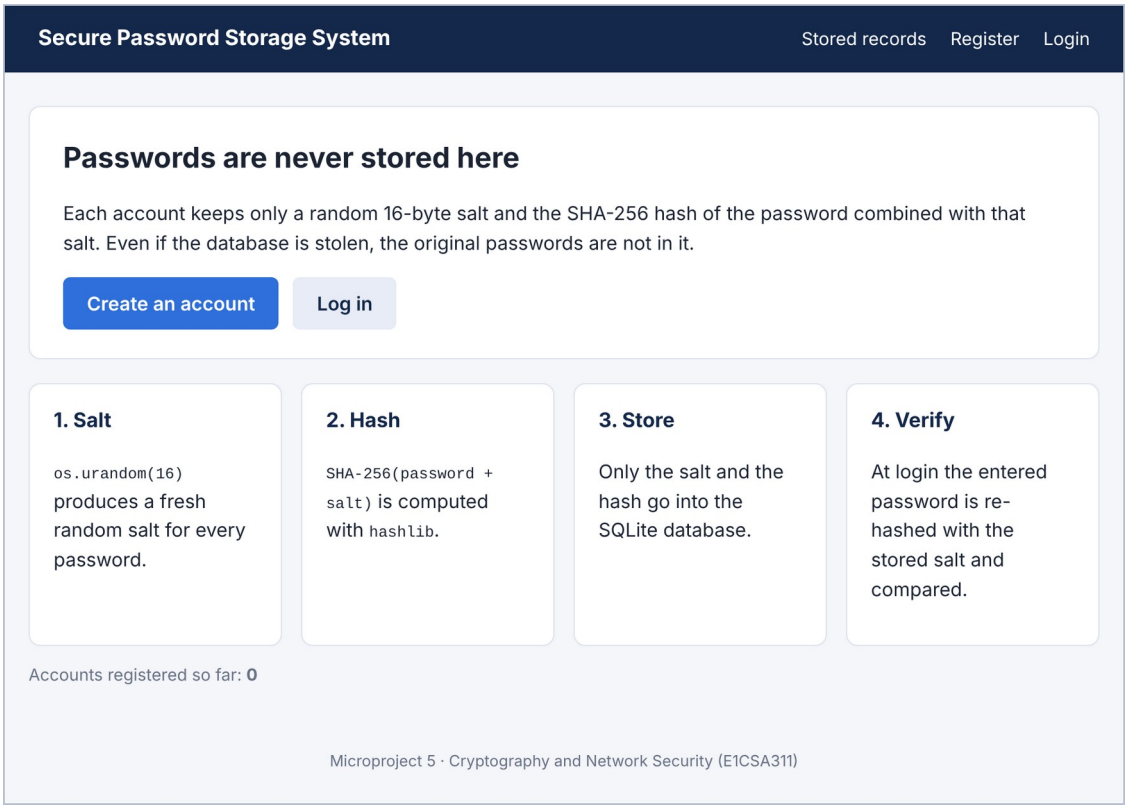


Figure 4.10: Web interface: home page

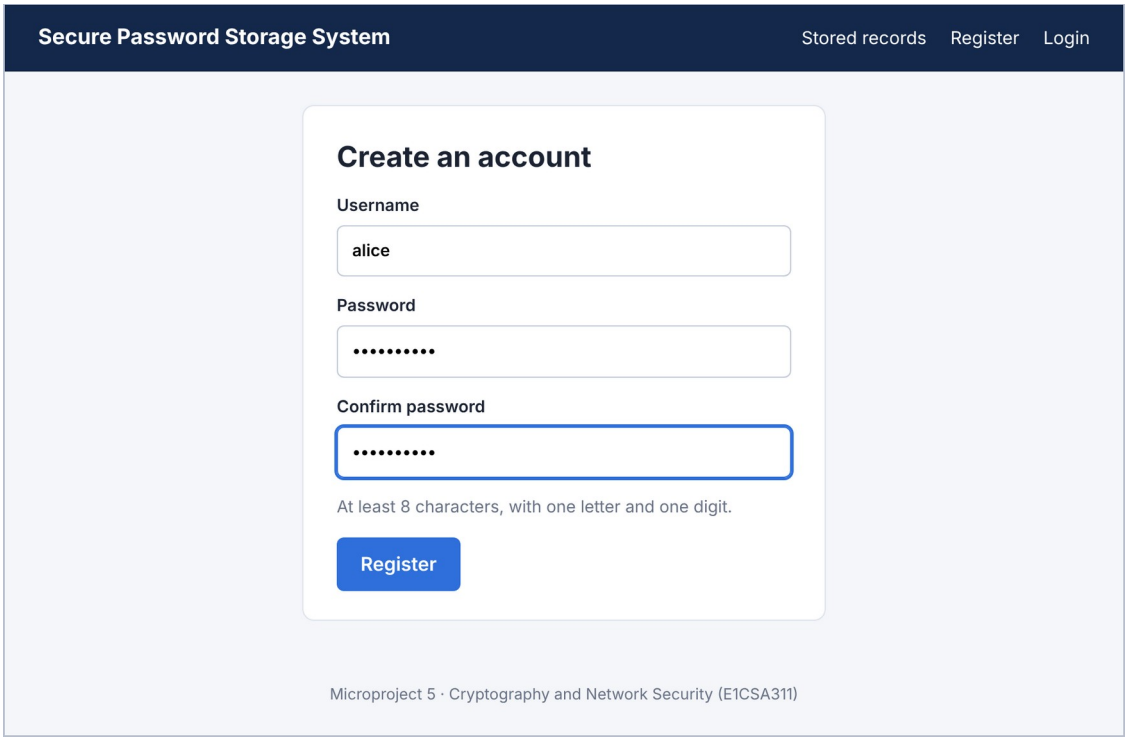


Figure 4.11: Web interface: registration form

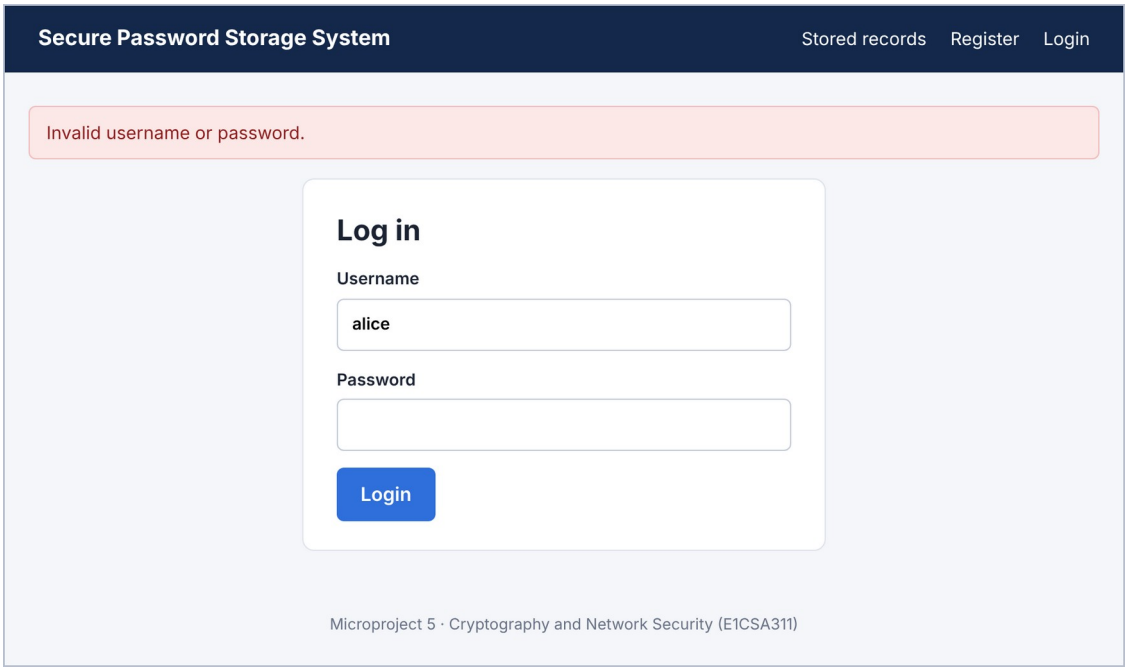


Figure 4.12: Web interface: failed login with a generic error message

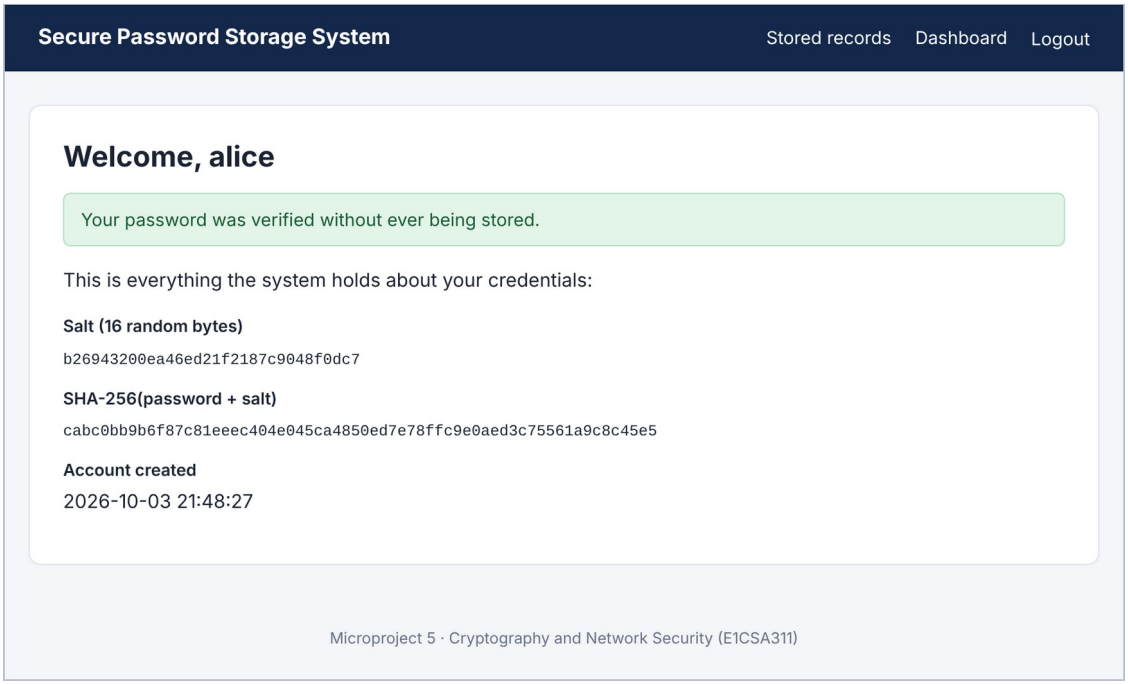


Figure 4.13: Web interface: dashboard after a successful login

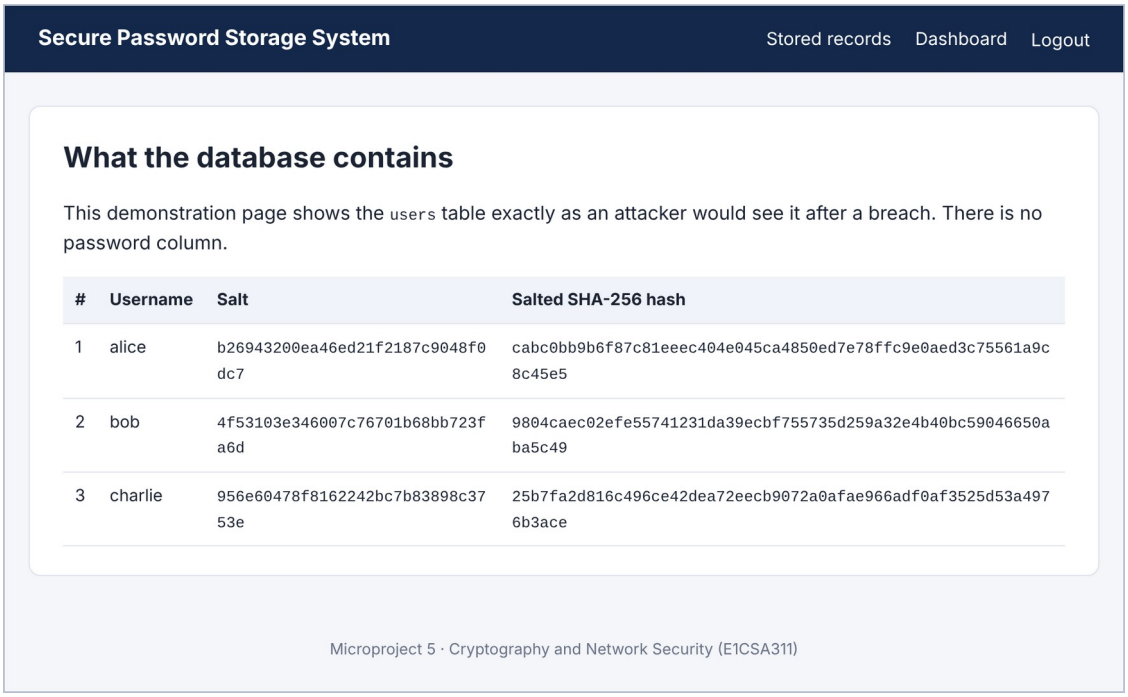


Figure 4.14: Web interface: contents of the users table (alice and bob have the same password)

5. CONCLUSION AND FUTURE SCOPE

Conclusion

This microproject designed and implemented a secure password storage system in Python. The system never stores a password. It generates a unique 128-bit salt for each password with `os.urandom()`, hashes the password together with the salt using SHA-256 from `hashlib`, stores only the salt and the hash in an SQLite database, and verifies a login by re-hashing the entered password with the stored salt.

The results confirm that the objectives were met. Users with the same password received different hashes; a precomputed lookup table that broke an unsalted database recovered nothing from the salted one; login verification worked correctly for valid, invalid and unknown credentials; and the plain-text password could not be found in the database file. The cryptographic work costs well under a microsecond per password, so the protection is effectively free for the application, while the cost of a dictionary attack on a stolen database grows in proportion to the number of accounts.

The project also showed where salted hashing stops: it removes precomputation and hides repeated passwords, but it does not make a single guess expensive. Understanding that boundary is as important as the implementation itself.

Future scope

- **Key stretching.** Replace the single SHA-256 with PBKDF2, bcrypt, scrypt or Argon2id so that each guess costs the attacker a significant amount of time and memory. Python already provides `hashlib.pbkdf2_hmac()` and `hashlib.scrypt()`.
- **Pepper.** Add a secret value kept outside the database (for example in an environment variable or a hardware security module), so that a database dump alone is not enough to start guessing.
- **Account lockout and rate limiting.** Limit repeated failed logins to slow down online guessing.
- **Transport security.** Serve the web interface over HTTPS so that the password is encrypted between the browser and the server.
- **Multi-factor authentication.** Add a one-time code as a second factor.
- **Stronger password policy.** Check new passwords against lists of breached passwords, as recommended by NIST SP 800-63B.
- **Algorithm upgrade path.** Store an algorithm identifier with each record so that hashes can be upgraded transparently when users next log in.

REFERENCES

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.

[2] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[3] P. A. Grassi et al., “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST Special Publication 800-63B, Jun. 2017.

[4] M. S. Turan, E. Barker, W. Burr and L. Chen, “Recommendation for Password-Based Key Derivation, Part 1: Storage Applications,” NIST Special Publication 800-132, Dec. 2010.

[5] R. Morris and K. Thompson, “Password security: A case history,” *Communications of the ACM*, vol. 22, no. 11, pp. 594–597, Nov. 1979.

[6] P. Oechslin, “Making a faster cryptanalytic time-memory trade-off,” in *Advances in Cryptology – CRYPTO 2003*, LNCS vol. 2729, Springer, 2003, pp. 617–630.

[7] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[8] Python Software Foundation, “hashlib — Secure hashes and message digests,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/hashlib.html

[9] Python Software Foundation, “os — Miscellaneous operating system interfaces (os.urandom),” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/os.html

[10] Python Software Foundation, “hmac — Keyed-hashing for message authentication,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/hmac.html

[11] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

[12] Pallets Projects, “Flask documentation.” [Online]. Available: https://flask.palletsprojects.com/

APPENDIX

A. How to run the project

Open the project folder in Visual Studio Code, open the integrated terminal and run:

```
python3 cli.py                                  # command-line interface
python3 demo.py                                 # security demonstrations
python3 benchmark.py                            # performance figures
python3 -m unittest discover -s tests -v        # unit tests

pip3 install -r requirements.txt                # Flask, for the web interface only
python3 app.py                                  # then open http://127.0.0.1:5000
```

The listings of the core module and the command-line interface follow. The remaining files (`app.py`, the HTML templates, `demo.py`, `benchmark.py` and the unit tests) are submitted in the project folder.

B. Source code: secure_store.py (core module)

```
"""
secure_store.py - core of the Secure Password Storage System.

Microproject 5, Cryptography and Network Security (E1CSA311).

What this module does
---------------------
* never stores a password in plain text
* generates a fresh random salt for every password with os.urandom()
* stores only  SHA-256(password || salt)  together with the salt
* verifies a login by re-hashing the entered password with the stored salt
  and comparing the two digests in constant time

Only the Python standard library is used (hashlib, os, hmac, sqlite3).
"""

import hashlib
import hmac
import os
import re
import sqlite3
from datetime import datetime

SALT_BYTES = 16          # 128-bit salt
MIN_PASSWORD_LENGTH = 8
USERNAME_PATTERN = re.compile(r"^[A-Za-z0-9_.-]{3,30}$")
DEFAULT_DB = os.path.join(os.path.dirname(os.path.abspath(__file__)), "users.db")


# ---------------------------------------------------------------- hashing --

def generate_salt(length=SALT_BYTES):
    """Return `length` cryptographically secure random bytes."""
    return os.urandom(length)


def hash_password(password, salt):
```

```
    """Return the hex SHA-256 digest of the password with the salt appended."""
    return hashlib.sha256(password.encode("utf-8") + salt).hexdigest()


def hash_unsalted(password):
    """Plain SHA-256 with no salt. Used only by the demos to show the weakness."""
    return hashlib.sha256(password.encode("utf-8")).hexdigest()


# ---------------------------------------------------------- validation --

def check_username(username):
    """Return an error message, or None if the username is acceptable."""
    if not USERNAME_PATTERN.match(username or ""):
        return "Username must be 3-30 characters: letters, digits, '_', '.' or '-'."
    return None


def check_password(password):
    """Return an error message, or None if the password meets the policy."""
    if len(password or "") < MIN_PASSWORD_LENGTH:
        return f"Password must be at least {MIN_PASSWORD_LENGTH} characters long."
    if not re.search(r"[A-Za-z]", password) or not re.search(r"\d", password):
        return "Password must contain at least one letter and one digit."
    return None


# ------------------------------------------------------------- storage --

class UserStore:
    """SQLite-backed table of (username, salt, salted hash)."""

    def __init__(self, db_path=DEFAULT_DB):
        self.db_path = db_path
        with self._connect() as con:
            con.execute(
                """CREATE TABLE IF NOT EXISTS users (
                       id            INTEGER PRIMARY KEY AUTOINCREMENT,
                       username      TEXT UNIQUE NOT NULL,
                       salt          TEXT NOT NULL,
                       password_hash TEXT NOT NULL,
                       created_at    TEXT NOT NULL
                   )"""
            )

    def _connect(self):
        con = sqlite3.connect(self.db_path)
        con.row_factory = sqlite3.Row
        return con

    def register(self, username, password):
        """Create an account. Returns (ok, message)."""
        username = (username or "").strip()
        error = check_username(username) or check_password(password)
        if error:
            return False, error

        salt = generate_salt()
```

```
        digest = hash_password(password, salt)
        try:
            with self._connect() as con:
                # parameterised query -> no SQL injection
                con.execute(
                    "INSERT INTO users (username, salt, password_hash, created_at) "
                    "VALUES (?, ?, ?, ?)",
                    (username, salt.hex(), digest,
                     datetime.now().strftime("%Y-%m-%d %H:%M:%S")),
                )
        except sqlite3.IntegrityError:
            return False, "That username is already taken."
        return True, "Account created."

    def verify(self, username, password):
        """Return True only if the username exists and the password matches."""
        with self._connect() as con:
            row = con.execute(
                "SELECT salt, password_hash FROM users WHERE username = ?",
                ((username or "").strip(),),
            ).fetchone()

        if row is None:
            # Do the same amount of work for an unknown user, so response
            # time does not reveal which usernames exist.
            hash_password(password or "", generate_salt())
            return False

        salt = bytes.fromhex(row["salt"])
        candidate = hash_password(password or "", salt)
        # constant-time comparison prevents timing attacks on the digest
        return hmac.compare_digest(candidate, row["password_hash"])

    def list_users(self):
        """Return every stored record. There is no password column to return."""
        with self._connect() as con:
            rows = con.execute(
                "SELECT id, username, salt, password_hash, created_at "
                "FROM users ORDER BY id"
            ).fetchall()
        return [dict(r) for r in rows]

    def delete_all(self):
        with self._connect() as con:
            con.execute("DELETE FROM users")
```

C. Source code: cli.py (command-line interface)

```
"""
cli.py - menu-driven terminal interface for the Secure Password Storage System.

Run:  python cli.py
"""

from getpass import getpass

from secure_store import UserStore
```

```
MENU = """
============================================
      SECURE PASSWORD STORAGE SYSTEM
============================================
  1. Register a new user
  2. Login
  3. View stored records (what the database holds)
  4. Exit
--------------------------------------------"""


def register(store):
    username = input("Choose a username : ")
    password = getpass("Choose a password : ")
    if password != getpass("Confirm password  : "):
        print("[!] Passwords do not match.")
        return
    ok, message = store.register(username, password)
    print(("[+] " if ok else "[!] ") + message)


def login(store):
    username = input("Username : ")
    password = getpass("Password : ")
    if store.verify(username, password):
        print(f"[+] Login successful. Welcome, {username}!")
    else:
        print("[!] Invalid username or password.")


def show_records(store):
    users = store.list_users()
    if not users:
        print("(no users registered yet)")
        return
    for u in users:
        print(f"\nUser #{u['id']}: {u['username']}   (created {u['created_at']})")
        print(f"  salt : {u['salt']}")
        print(f"  hash : {u['password_hash']}")
    print("\nNote: the original passwords are not stored anywhere.")


def main():
    store = UserStore()
    actions = {"1": register, "2": login, "3": show_records}
    while True:
        print(MENU)
        choice = input("Enter your choice: ").strip()
        if choice == "4":
            print("Goodbye.")
            break
        action = actions.get(choice)
        if action:
            action(store)
        else:
            print("[!] Please enter a number from 1 to 4.")
```

```
if __name__ == "__main__":
    main()
```